

Komponen Utama IoT dan Fungsinya

- Sensing & Embedding Components**
 - Sensor: menangkap / mengukur perubahan data fisik (suhu, cahaya, gas, jarak).
 - Actuator: melakukan aksi (gerakkan motor, nyalakan lampu).
- Connectivity**
 - Media komunikasi: WiFi, Bluetooth, Zigbee, LoRa, 4G/5G, MQTT.
 - Fungsi: kirim data antar perangkat → cloud.
- IoT Cloud**
 - Menyimpan, mengolah, dan menganalisis data dari perangkat IoT. (AWS IoT, Microsoft Azure IoT.)
- IoT Analytics & Data Management**
 - Mengolah data mentah → informasi.
 - Prediksi, deteksi anomali, automasi. (Big Data, Machine Learning untuk prediksi)
- End-user Devices & UI**
 - Perangkat yg digunakan user untuk monitoring & kontrol. (Aplikasi mobile, web dashboard.)

Perbedaan Microprocessor dan Microcontroller

- Microprocessor
- Chip pemrosesan utama yang membutuhkan perangkat tambahan seperti RAM, ROM, dan I/O eksternal.
Contoh Device: Intel Core i5, AMD Ryzen, Raspberry Pi (CPU-nya microprocessor).
Memori: Tidak memiliki memori internal, RAM/ROM terpisah. Jadi butuh memori eksternal untuk ops.
Kapasitas Komputasi: Sangat tinggi, bisa menjalankan OS kompleks.
Kecepatan Clock: Tinggi, dari 1 GHz – 5 GHz.
Sistem Operasi: Windows, Linux, MacOS.
Use Case: Laptop, PC, sistem yang butuh komputasi berat.

- Microcontroller
- Sistem komputer mini dalam satu chip (CPU + RAM + ROM + I/O).
Contoh Device: Arduino (ATmega328), ESP32, STM32.
Memori: Tersedia memori internal (Flash + RAM).
Kapasitas Komputasi: Rendah–sedang, tergantung arsitektur. Biasanya digunakan dalam perangkat embedded.
Kecepatan Clock: 8 MHz – 240 MHz.
Sistem Operasi: Bare metal, FreeRTOS, atau tanpa OS.
Use Case: IoT, smart home, sensor, robot sederhana.

Contoh Sensor dan Fungsinya

- Sensor - Fungsi
- DHT22 / LM35 = Mengukur suhu (dan kelembaban)
- LDR = Mengukur intensitas cahaya
- Ultrasonic HC-SR04 = Mengukur jarak
- MQ-2 = Deteksi gas/asap
- PIR = Deteksi gerakan
- Load cell = Mengukur berat

Cara Kerja Sensor Ultrasonic (HC-SR04)

Sensor ultrasonik bekerja dengan cara mengirimkan gelombang suara frekuensi tinggi (di luar rentang pendengaran manusia) yang memantul kembali setelah mengenai objek. Dengan mengukur waktu yang diperlukan gelombang suara untuk kembali, sensor dapat menghitung jarak objek.

- Fungsi: Mengukur jarak atau mendeteksi objek.
- Penggunaan: Digunakan dalam robotik, sistem parkir otomatis, dan pengukuran level cairan.

Peranan IoT di Industri

- Industri Kesehatan**
Monitoring pasien real-time (HR, SpO2).
Tracking alat medis.
 - Contoh: gelang IoT kirim detak jantung → dokter → respon cepat.
- Industri Pertanian**
Smart irrigation (penyiraman otomatis).
Monitoring kelembaban tanah, cuaca.
 - Contoh: sensor tanah aktifkan pompa saat tanah kering.

IoT Components (AWS tech stack)

- Message Broker:**
Fungsi: Mengatur pertukaran pesan antara perangkat IoT dan server seperti AWS.
Analogi: Seperti kantor pos yang mengirim dan menerima pesan antara pengirim dan penerima.
Contoh Protokol: MQTT.
 - Thing Registry:**
Fungsi: Daftar resmi semua perangkat IoT yang terhubung, memberikan identitas pada perangkat (misalnya sensor suhu atau kamera).
Analogi: Seperti daftar nama perangkat yang terdaftar di sistem, agar sistem tahu perangkat mana yang aktif.
 - Thing Shadows:**
Fungsi: Menyimpan status virtual perangkat IoT, memungkinkan AWS mengetahui status perangkat meskipun perangkat tersebut offline.
Analogi: Bayangan perangkat yang selalu menunjukkan status terakhir perangkat.
 - Rules Engine:**
Fungsi: Memproses data dari perangkat IoT dan menjalankan aturan otomatis berdasarkan kondisi tertentu (misalnya mengirim notifikasi jika suhu melebihi 40°C).
Analogi: Otak dari sistem IoT yang memutuskan tindakan yang harus diambil berdasarkan data yang diterima.

Konsep Ekosistem IoT untuk Industri

Pternakan Ayam

- Tujuan:
Monitoring suhu, pakan, air, kualitas udara, feeding otomatis, monitoring via HP.
- Sensing Layer**
 - DHT22 → suhu/kelembaban
 - Load cell → berat pakan
 - Water level → ketinggian air
 - MQ-135 → kualitas udara
 - Kamera → kondisi kandang
 - Connectivity Layer**
 - WiFi / LoRa
 - MQTT (ringan & populer untuk IoT)
 - Edge/Embedded Layer**
 - ESP32 / STM32 per kandang
 - Membaca sensor + control actuator
 - Kirim data → cloud
 - Actuators**
 - Kipas otomatis
 - Lampu pemanas
 - Motor servo pemberi pakan
 - Pompa air
 - Cloud Layer**
 - Menyimpan data historis
 - Analitik & notifikasi
 - Automasi (jika suhu tinggi → kipas ON)
 - Application Layer**
 - Dashboard HP/laptop:
 - Grafik suhu
 - Stok makanan
 - Status tiap kandang
 - Kontrol jarak jauh

Cloud protocol adalah protokol komunikasi yang digunakan oleh perangkat IoT untuk mengirim dan menerima data ke/dari layanan cloud (server). Setiap protokol memiliki karakteristik, efisiensi, dan tujuan penggunaan yang berbeda tergantung kebutuhan sistem IoT (misalnya konsumsi daya, ukuran data, atau kecepatan komunikasi).

Enkripsi

- Symmetric Encryption:** Menggunakan kunci yang sama untuk enkripsi dan dekripsi (misalnya AES, DES).
Kelebihan: Cepat dan efisien.
Kekurangan: Kurang aman untuk komunikasi jarak jauh karena kunci yang sama digunakan untuk enkripsi dan dekripsi.
- Asymmetric Encryption (Public-Private Key):** Menggunakan dua kunci berbeda; kunci publik untuk enkripsi dan kunci pribadi untuk dekripsi (misalnya RSA, PGP).

Kelebihan: Lebih aman karena kunci pribadi tidak pernah dibagikan.
Kekurangan: Lebih lambat dibandingkan dengan symmetric encryption.

- Cryptographic Hash:** Menghasilkan kode unik (hash) dari data, sering digunakan untuk verifikasi integritas file dan tanda tangan digital (misalnya MD5, SHA-256).

Kelebihan: Sangat aman dan tidak dapat didekripsi kembali.

Protokol IoT

- MQTT (Message Queuing Telemetry Transport):**
Protokol ringan dan efisien, cocok untuk perangkat dengan daya terbatas. Menggunakan model publish-subscribe dimana perangkat yang mengirim data (publisher) akan mengirimkan pesan ke topik yang telah ditentukan, dan perangkat yang berlangganan (subscriber) akan menerima pesan tersebut. Ada broker (server) yang bertugas menerima dan mendistribusikan pesan.
Kelebihan: Cepat, hemat daya, cocok untuk jaringan tidak stabil.
Kekurangan: Tidak cocok untuk transfer data besar.
Contoh Penggunaan:
 - Sensor lingkungan (suhu, kelembaban, tekanan udara).
 - Smart home (lampu, AC, pintu otomatis).
 - Monitoring industri.
- HTTP/HTTPS:**

Protokol standar yang digunakan untuk komunikasi berbasis web. Dalam IoT, digunakan untuk perangkat mengirim data ke server melalui REST API. Perangkat IoT mengirim request (permintaan) ke server, dan server mengirim response (balasan).
Kelebihan: Kompatibel dengan banyak platform dan bahasa pemrograman.
Kekurangan: Tidak efisien untuk komunikasi real-time, menggunakan bandwidth lebih banyak.
Contoh Penggunaan:

- Smart meter (pengiriman data berkala ke server).
- Integrasi IoT dengan layanan web atau dashboard berbasis API.

- CoAP (Constrained Application Protocol):**
Protokol berbasis UDP yang lebih efisien dan cepat dibanding HTTP. Cocok untuk perangkat dengan sumber daya terbatas.
Kelebihan: Lebih ringan dari HTTP, bekerja dengan baik di jaringan tidak stabil.
Kekurangan: Tidak seanggih HTTPS dari sisi keamanan.
Contoh Penggunaan:
 - Sistem smart agriculture (sensor tanah, kelembaban).
 - Perangkat IoT di jaringan lokal (misalnya rumah pintar).
- AMQP (Advanced Message Queuing Protocol):**

Digunakan untuk aplikasi yang membutuhkan pengiriman pesan yang andal dan memiliki keamanan tinggi (misalnya IoT industri).
Cara Kerja:

- Menggunakan message broker seperti RabbitMQ.
- Mendukung queue (antrian), routing, dan acknowledgment (konfirmasi pesan diterima).

 Kelebihan: Pesan tidak hilang meski jaringan terputus, mendukung keamanan tinggi.
Kekurangan: Kompleks, memerlukan daya dan memori lebih besar dibanding MQTT.
Contoh Penggunaan:

- Sistem logistik atau manufaktur besar.
- IoT industri (IIoT) yang memerlukan jaminan pengiriman data.

- WebSocket:**
Protokol komunikasi dua arah yang memungkinkan pertukaran data secara real-time antara client dan server melalui satu koneksi TCP.
Cara Kerja:
 - Setelah koneksi dibuka, data bisa dikirim bolak-balik kapan saja, tanpa harus mengirim request baru seperti HTTP.
- Kelebihan: Latensi rendah, efisien untuk komunikasi yang berlangsung terus-menerus (misalnya streaming data).
- Kekurangan: Kompleks dalam implementasi, membutuhkan koneksi internet yang stabil.
- Contoh Penggunaan:
 - Aplikasi monitoring real-time (dashboard suhu, GPS tracking).
 - Sistem notifikasi instan.
 - IoT berbasis kontrol langsung (misalnya robotik jarak jauh).

Security Best Practice for MQTT

- Gunakan versi OS dan Library yang terbaru karena bisa jadi ada patch buat celah keamanan.
- Gunakan hardware yang punya fitur keamanan seperti Trusted Execution, Environments, Trusted Platform Modules, dan non-executive spaces.
- Obfuscating atau “mengaburkan” code itu secara relative tidak berguna. Lebih baik kita kasih Signature, Encrypt, dan jaga firmware dan software image kita.
- Jangan pake default password suatu perangkat.
- Gunakan Root of Trust dan Secure Boot untuk memastikan kita punya “Golden” image dari software yang running di device pengguna.
- Jangan pake hardcoded password di ROM images.
- Semua Port IP harus mati by default.
- Pake Address Space Layout Randomization, Stack Canaries, Guard Bands melalui OS modern.
- Pikirln kalau perangkat kita mau End of Life, persistent memory itu harus di wipe sebelum dibuang.
- Gunakan sistem bug bounty buat nyari bugs.
- Subscribe dan partisipasi dalam US-CERT active threat management. Ketika mau rilis produk, pastikan dia menggunakan TLS atau DTLS. Encrypt data dari sensor ke cloud.
- Gunakan Anti-Debug Fuse. Intinya dengan pake ini, orang gabisa ngedebug produk kita.

IoT Components Sensor & Power

Sensor pada IoT

a. Thermocouples (Sensor Suhu)

- Mengukur suhu berdasarkan tegangan listrik dari dua logam berbeda.
- Tidak memerlukan sumber daya tambahan.
- Kelebihan: Sederhana, cocok untuk suhu tinggi.
- Kekurangan: Akurasi bisa terpengaruh oleh kualitas logam.

b. RTD (Resistance Temperature Detector)

- Mengukur suhu berdasarkan perubahan resistansi logam (biasanya platinum).
- Kelebihan: Akurat untuk suhu rendah–menengah (<600°C).
- Kekurangan: Tidak cocok untuk suhu tinggi karena bisa terkontaminasi.

c. Thermistor

- Mirip dengan RTD, tetapi berbasis keramik/polimer.
- Hubungan resistansi dan suhu tidak linear.

• Jenis:

- o NTC: resistansi turun saat suhu naik.
- o PTC: resistansi naik saat suhu naik.
- Kelebihan: Sensitif untuk rentang suhu sempit.

d. Sensor Cahaya (Photoresistor, Photodiode)

- Photoresistor: resistansi berubah tergantung intensitas cahaya.
- Photodiode: mengubah cahaya menjadi arus listrik.
- Digunakan pada lampu otomatis, sistem keamanan, dan smart lighting.

e. PIR Sensor (Pyroelectric Infrared)

- Mendeteksi gerakan tubuh hangat (radiasi inframerah).
- Contoh: alarm keamanan atau lampu otomatis yang menyala saat ada orang.

f. LiDAR (Light Detection and Ranging)

- Sensor aktif yang memancarkan sinar laser untuk mengukur jarak.
- Digunakan pada: mobil tanpa pengemudi, drone, robot, dan pertanian presisi.

g. MEMS Sensors (Micro-Electro-Mechanical Systems)

- Sensor mikro untuk mendeteksi gerakan, getaran, atau suara.
- o Accelerometer: mendeteksi percepatan linier.
- o Gyroscope: mendeteksi rotasi.
- o MEMS Microphone: menangkap getaran atau suara.
- Banyak digunakan di industri dan sistem prediksi kerusakan mesin.

h. Smart IoT Endpoints (Sensor Cerdas)

- Sensor dengan prosesor dan pengolahan data internal, misalnya: o Kamera (CCD, CMOS)
- o Sistem visi (vision system)
- Mampu menganalisis data secara lokal sebelum dikirim ke cloud.

i. Input & Output Devices

- Input: data dari sensor (gas, kelembapan, radiasi, suara, ultrasonik, dll).
- Output: perangkat yang menanggapi data, seperti lampu LED, motor, katup, speaker, atau sistem video.

2. Power pada IoT

A. Konsumsi daya pada sensor

contoh : TI SensorTag c2650

B. Energi Harvesting (Pemanfaatan energi lingkungan) -->

Mengubah energi di sekitar (panas, cahaya, gelombang radio) menjadi energi listrik untuk mengisi daya sensor IoT.

- Solar Harvesting: energi dari cahaya matahari atau lampu.
- RF Energy Harvesting: energi dari sinyal radio.
- Thermal Harvesting: energi dari perbedaan suhu.

Kelebihan:

- Ramah lingkungan.
- Tidak perlu penggantian baterai sering.
- Cocok untuk perangkat daya rendah

C. Baterai dan Supercapacitor

1. Baterai Lithium-Ion (Li-ion) --> Umum digunakan karena kepadatan energi tinggi. Energi dihasilkan dari perpindahan ion lithium antara elektroda.

Kelemahan:

- o Daya berkurang seiring waktu (sekitar 30% setelah 1000 kali isi ulang).
- o Sensitif terhadap suhu tinggi (bisa rusak).
- o Perlu manajemen panas yang baik.

2. Supercapacitor --> Menyimpan energi secara elektrostatis, bukan kimia. Densitas energi 1–10 Wh/kg (lebih tinggi dari kapasitor biasa).

Kelebihan:

- o Pengisian sangat cepat (dalam hitungan detik).
- o Tidak bisa overcharge.

Kekurangan:

- o Mahal (lebih dari \$10.000).
- o Kapasitas masih lebih kecil dari baterai (200 Wh/kg).

Embedded System

Definisi: Sistem komputer yang dirancang untuk tujuan tertentu, biasanya terkandung dalam perangkat lain, seperti mikrokontroler yang dilengkapi dengan memori dan I/O.

Komponen: Mikrokontroler, firmware, dan RTOS (Real-Time Operating System) yang mengatur proses waktu nyata.

Desain IoT Solusi

Sistem IoT ini menggunakan sensor suhu, broker MQTT, dan dua perangkat penerima data: Temperature Controller dan Phone. Semua proses berbasis metode publish–subscribe MQTT.

Alur Kerja:

1. Sensor Suhu membaca suhu (misalnya 80°C) secara berkala dan mengirimkan data ke MQTT Broker dengan topik “Topic/Temperature”.
 2. MQTT Broker menerima dan menyebarkan data ke perangkat yang berlangganan topik tersebut, yaitu Temperature Controller dan Phone.
 3. Temperature Controller membandingkan suhu dengan batas aman dan dapat mengaktifkan tindakan seperti menyalakan kipas atau mematikan mesin jika suhu tinggi.
 4. Phone menampilkan data suhu kepada pengguna dan memberikan notifikasi jika suhu melebihi batas aman.
- Dengan sistem ini, seluruh perangkat terhubung secara real-time dan responsif melalui MQTT.

IoT Security Attack and Threat

Jenis Serangan	Penjelasan	Dampak
Amplification Attack	Menyerang server dengan memperbanyak lalu lintas data (bandwidth) dari sumber sah seperti DNS atau NTP untuk membanjiri korban.	Server jadi lambat atau mati.
ARP Spoofing	Menipu jaringan agar alamat IP korban diarahkan ke penyerang.	Data bisa disadap.
Banner Scanning	Menyusuri sistem untuk melihat versi OS atau aplikasi target.	Membantu hacker mengenali celah.
Botnet	Sekelompok perangkat IoT yang terinfeksi malware dan dikendalikan dari jarak jauh.	Digunakan untuk DDoS, spam, dan spyware.
Brute Force / Dictionary Attack	Menebak password secara berulang dengan mencoba semua kemungkinan atau daftar kata.	Password mudah bisa dibobol.
Buffer Overflow	Mengisi memori lebih dari kapasitasnya hingga bisa menulis kode jahat ke sistem.	Menyebabkan sistem crash atau dikendalikan.
Man-in-the-Middle (MITM)	Penyerang menyusup di antara dua pihak yang berkomunikasi dan mencuri data mereka.	Data pribadi bisa dicuri.
RCE (Remote Code Execution)	Penyerang menjalankan kode berbahaya di perangkat dari jarak jauh.	Perangkat bisa sepenuhnya diambil alih.
SYN Flood / DDoS	Membanjiri server dengan permintaan koneksi palsu hingga tidak bisa merespons.	Sistem jadi tidak tersedia.
Zero-Day Exploit	Serangan yang memanfaatkan celah keamanan yang belum diketahui pembuatnya.	Sangat berbahaya karena belum ada perbaikan.
Fuzzing Attack	Mengirim data aneh untuk melihat bagaimana perangkat bereaksi.	Dapat mengungkap bug tersembunyi.

IoT Security Defense

Metode Pertahanan	Penjelasan
ASLR (Address Space Layout Randomization)	Menyusun ulang posisi memori program secara acak agar hacker sulit menebak lokasi kode penting.
DEP (Data Execution Prevention)	Melarang bagian memori tertentu untuk menjalankan kode. Mencegah eksekusi kode jahat dari buffer overflow.
Blackhole / Sinkhole	Mengarahkan lalu lintas berbahaya ke alamat “kosong” agar tidak merusak sistem utama.
DPI (Deep Packet Inspection)	Menganalisis setiap paket data untuk mencari virus, spam, atau penyusup.
Firewall	Menyaring data masuk dan keluar antara jaringan aman dan tidak aman berdasarkan aturan tertentu.
Honeypot	“Umpan palsu” yang menyerupai sistem asli untuk menjebak hacker dan mempelajari cara serangannya.
IDS (Intrusion Detection System)	Mendeteksi adanya ancaman di jaringan tanpa mengganggu lalu lintas utama.
IPS (Intrusion Prevention System)	Mendeteksi dan langsung memblokir ancaman yang masuk ke jaringan.
PKI (Public Key Infrastructure)	Sistem keamanan berbasis sertifikat digital untuk memastikan identitas pengirim data.
Private & Public Key	Public key untuk mengenkripsi data; private key untuk mendekripsinya. Hanya pemilik private key yang bisa membuka pesan.

Arsitektur Komunikasi Dasar

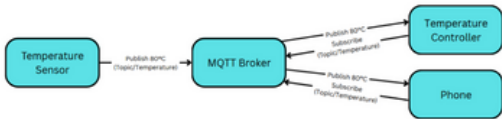
Data yang ditransmisikan bisa mengalami kerusakan atau bit error, yang diukur dengan Bit Error Rate (BER) (jumlah bit rusak dibagi total bit).

Beberapa penyebab bit error adalah:

1. Channel Noise
2. Kualitas media transmisi mempengaruhi data. Misalnya, kabel fiber optic yang rusak bisa menyebabkan cahaya tidak tersalur dengan baik, merusak data.
3. Interference
4. Medan elektromagnetik (EMF) dari kabel listrik bisa merusak data di kabel LAN. Solusinya: Jauhkan kabel LAN dari sumber interferensi kuat.
5. Multipath Fading
6. Jika kabel terlalu panjang, arus bisa melemah, menyebabkan bit error. Solusinya seperti repeater di Minecraft, yang memperkuat sinyal yang melemah.
7. Attenuation
8. Kesalahan pada modulasi sinyal juga bisa menyebabkan bit error. Solusi: Meningkatkan daya untuk mengatasi pelemahan sinyal.

Solusi untuk mengurangi bit error:

- Pilih media transmisi berkualitas tinggi.
- Perbaiki receiver agar lebih sensitif.
- Gunakan Forward Error Correction (FEC) untuk menambah informasi yang membantu mengatasi kesalahan.



Tipe Komunikasi dan Routing

Komunikasi bisa bermacam-macam dan dikombinasikan sesuai kebutuhan. Untuk mengatur jalur data, digunakan routing dan gateway, yang memiliki fungsi:

1. Internet Gateway: Menghubungkan cluster ke internet.
2. Fail-Over: Beralih ke gateway cadangan jika yang utama gagal.
3. VLAN: Pengelompokan jaringan.
4. VPN: Membuat jalur aman antara titik.
5. Encryption: Keamanan data.
6. QoS: Manajemen kualitas dan kecepatan data.

Router dibagi menjadi dua:

- Data Plane: Penerima dan pengirim data.
- Control Plane: Mengatur rute dan cara pengiriman data.

Jika jaringan sibuk, router bisa kewalahan. Solusinya, dengan Software Defined Network (SDN), Data Plane dan Control Plane dipisah. SDN bertugas mengelola rute dan pengiriman, sementara perangkat routing hanya menerima tugas.

Layer SDN:

1. Infrastructure Layer: Perangkat fisik seperti router dan sensor.
2. Control Layer: Mengelola routing dan pengiriman data.
3. Application Layer: Mengatur kebijakan routing.

API dalam SDN:

- Southbound API (misalnya OpenFlow) menghubungkan Control Layer ke Infrastructure Layer.
- Northbound API (misalnya REST API) menghubungkan Control Layer ke Application Layer.

Cara kerja MQTT

1. Subscriber : Menerima message
2. Publisher : Mempublish message
3. Broker : Perantara yang menerima message dari Publisher dan mengirim ke Subscriber
4. Topics : Tempat atau tujuan atau sumber suatu message

Contoh:

Publisher mengirim message ke Topic topic/contoh. Lalu Broker menerima message dan forward message tersebut ke Subscriber yang subscribe ke Topic topic/contoh.

